

Android Application Penetration Testing Report using Burp Suite and InsecureBankv2

Cyber Security Internship Project

Table of Contents

1. Introduction
 2. Lab Setup
 3. Vulnerability Testing
 4. Summary Table
 5. Conclusion
 6. References
-

1. Introduction

1.1 Purpose of the Project

Mobile applications frequently handle sensitive information such as user credentials, financial details, and personal data. Due to the increasing use of smartphones for banking and online services, ensuring application security has become essential. The purpose of this project is to perform penetration testing on a deliberately vulnerable Android application named **InsecureBankv2** to understand common security weaknesses and how they can be identified using industry-standard security tools.

The project focuses on configuring a mobile penetration testing environment using **Genymotion**, **Burp Suite Community Edition**, and a **Python Flask backend server**. During testing, HTTP requests exchanged between the Android application and the backend server were intercepted and analyzed to observe insecure communication and information disclosure.

1.2 Android Application Security

Android applications generally communicate with backend servers using HTTP or HTTPS protocols. If these communications are not properly secured, attackers may intercept sensitive information such as usernames, passwords, session tokens, and personal data.

Mobile penetration testing helps security professionals identify such weaknesses before attackers can exploit them. The testing process includes configuring an emulator, intercepting application traffic, analyzing requests and responses, and documenting security findings.

1.3 OWASP Mobile Top 5

The OWASP Mobile Top 5 highlights the most common security risks in Android and iOS applications.

M1 – Improper Credential Usage

Applications should securely manage user credentials and avoid exposing authentication data during transmission or storage.

M2 – Inadequate Supply Chain Security

Applications should use trusted third-party libraries and ensure dependencies are updated to prevent inherited vulnerabilities.

M3 – Insecure Authentication and Authorization

Weak authentication mechanisms or improper authorization checks may allow attackers to gain unauthorized access.

M4 – Insufficient Input and Output Validation

Applications should properly validate all user input and server responses to prevent injection attacks and unexpected behavior.

M5 – Insecure Communication

Sensitive information transmitted over unsecured channels such as HTTP can be intercepted by attackers. Secure communication should always use HTTPS with proper certificate validation.

2. Lab Setup

2.1 Tools Used

Tool	Purpose
Kali Linux	Penetration Testing Operating System
Genymotion Desktop	Android Emulator
Samsung Galaxy S3 Virtual Device	Android Testing Device
Android Debug Bridge (ADB)	APK Installation
Burp Suite Community Edition	HTTP Request Interception
Python Flask	Backend Server
InsecureBankv2	Vulnerable Android Banking Application

2.2 Testing Environment

The testing environment consisted of a Kali Linux virtual machine containing Burp Suite, Python, and the vulnerable application files. Genymotion Desktop was used to emulate an Android device. The Flask backend server hosted the vulnerable web services while Burp Suite acted as an interception proxy between the Android application and the backend.

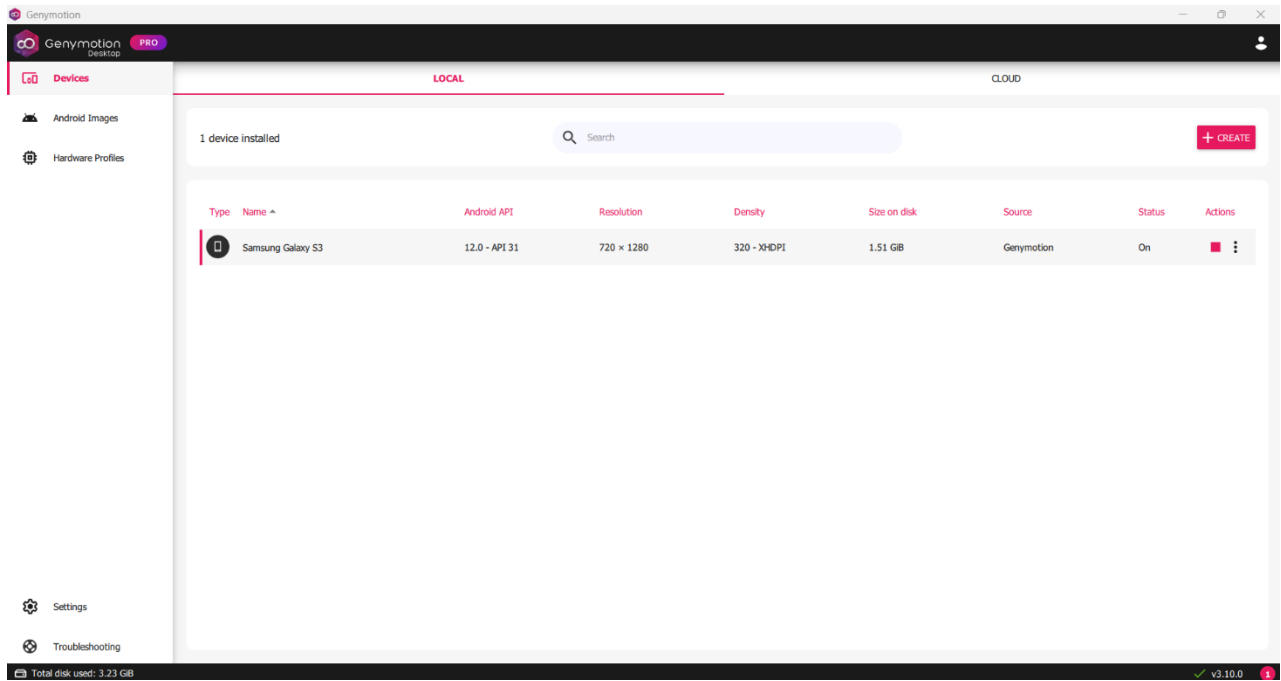


Figure 2.1: Samsung Galaxy S3 virtual device created in Genymotion.

2.3 Android Emulator Setup

A Samsung Galaxy S3 virtual device running Android 12 (API Level 31) was created using Genymotion Desktop. The emulator successfully booted and provided an isolated environment for penetration testing without requiring a physical Android device.

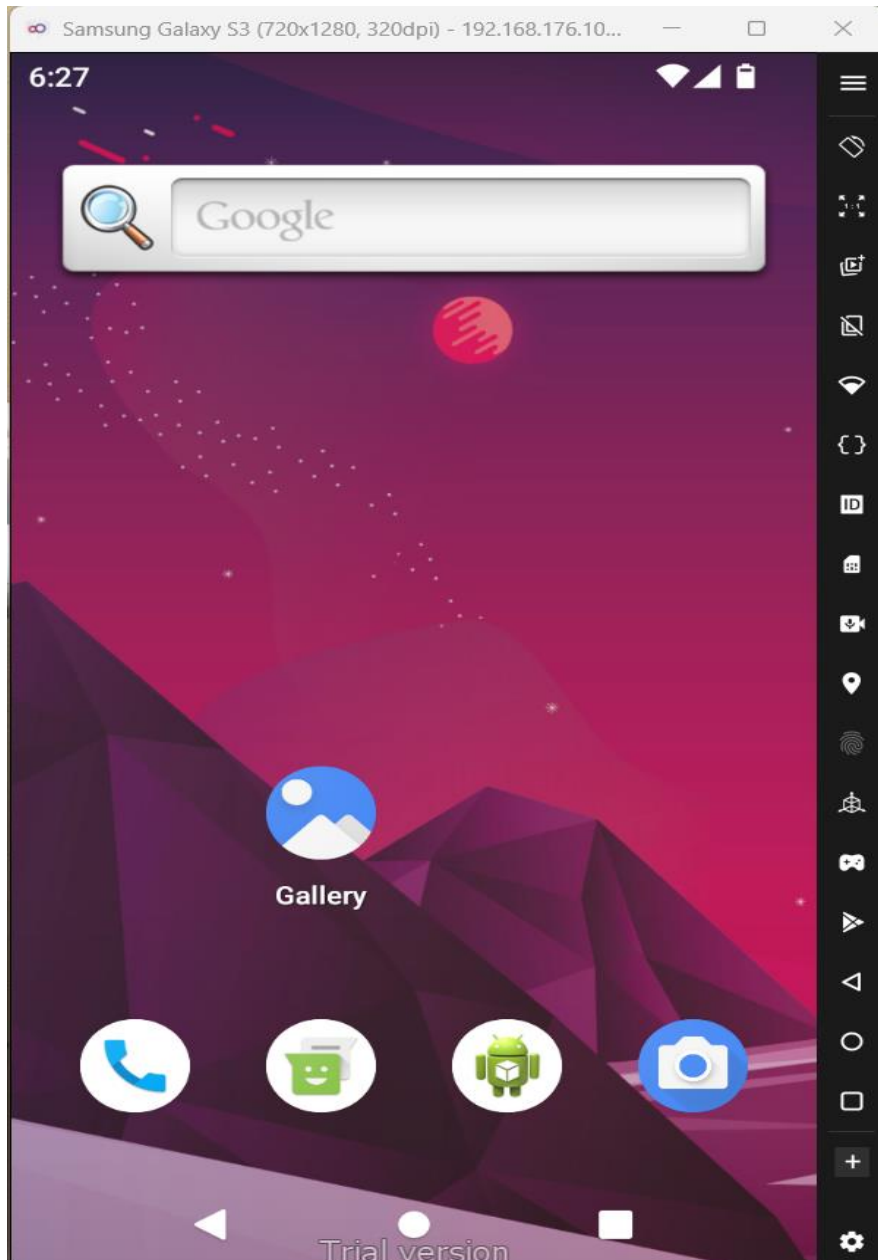


Figure 2.2: Android emulator running successfully.

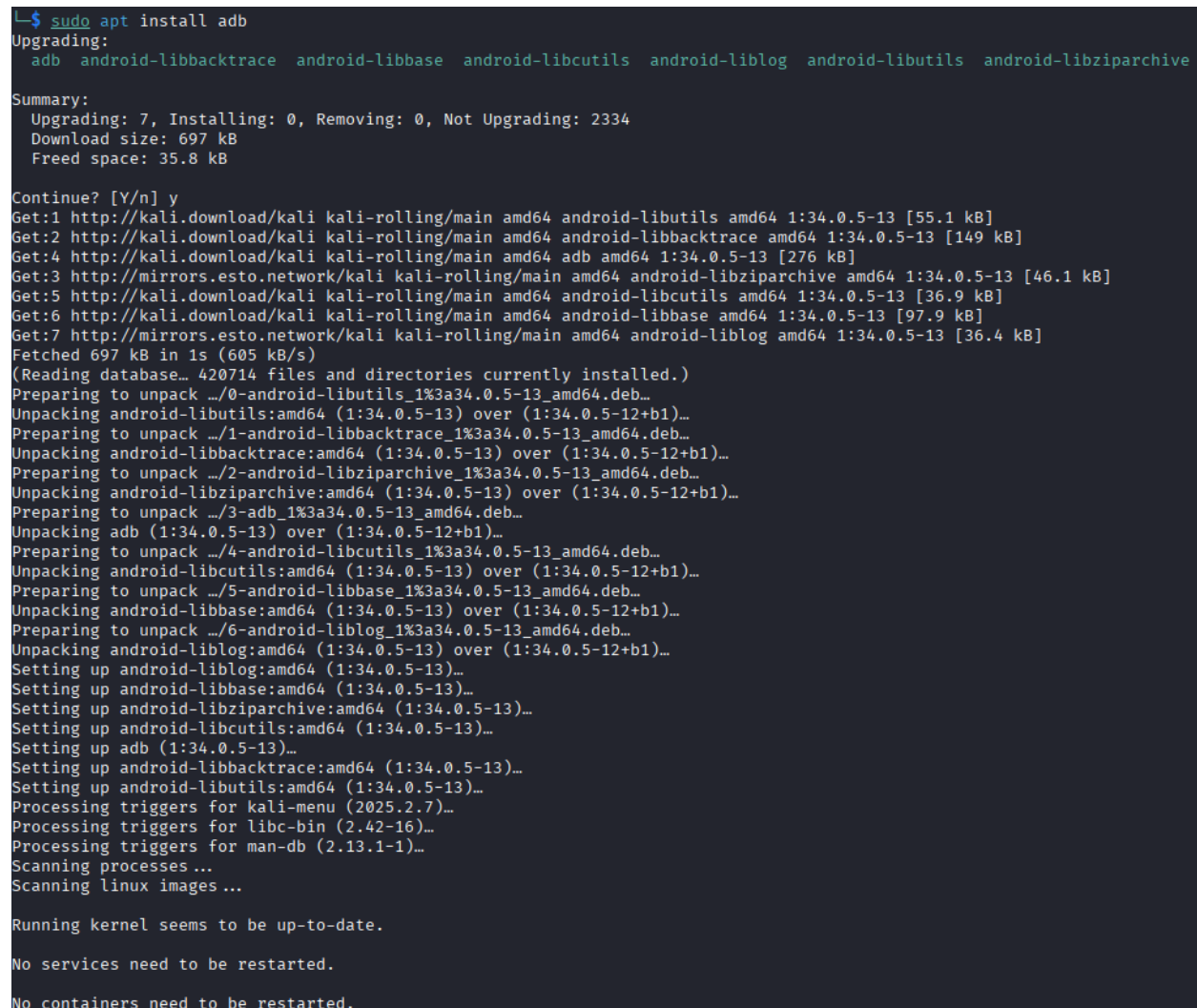
2.4 Installing the Vulnerable Application

The vulnerable Android application **InsecureBankv2.apk** was installed using Android Debug Bridge (ADB).

The following command was executed:

```
adb install InsecureBankv2.apk
```

The installation completed successfully.



```
└─$ sudo apt install adb
Upgrading:
  adb  android-libbacktrace  android-libbase  android-libcutils  android-liblog  android-libutils  android-libziparchive

Summary:
  Upgrading: 7, Installing: 0, Removing: 0, Not Upgrading: 2334
  Download size: 697 kB
  Freed space: 35.8 kB

Continue? [Y/n] y
Get:1 http://kali.download/kali kali-rolling/main amd64 android-libutils amd64 1:34.0.5-13 [55.1 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 android-libbacktrace amd64 1:34.0.5-13 [149 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 adb amd64 1:34.0.5-13 [276 kB]
Get:3 http://mirrors.esto.network/kali kali-rolling/main amd64 android-libziparchive amd64 1:34.0.5-13 [46.1 kB]
Get:5 http://kali.download/kali kali-rolling/main amd64 android-libcutils amd64 1:34.0.5-13 [36.9 kB]
Get:6 http://kali.download/kali kali-rolling/main amd64 android-libbase amd64 1:34.0.5-13 [97.9 kB]
Get:7 http://mirrors.esto.network/kali kali-rolling/main amd64 android-liblog amd64 1:34.0.5-13 [36.4 kB]
Fetched 697 kB in 1s (605 kB/s)
(Reading database... 420714 files and directories currently installed.)
Preparing to unpack .../0-android-libutils_1%3a34.0.5-13_amd64.deb...
Unpacking android-libutils:amd64 (1:34.0.5-13) over (1:34.0.5-12+b1)...
Preparing to unpack .../1-android-libbacktrace_1%3a34.0.5-13_amd64.deb...
Unpacking android-libbacktrace:amd64 (1:34.0.5-13) over (1:34.0.5-12+b1)...
Preparing to unpack .../2-android-libziparchive_1%3a34.0.5-13_amd64.deb...
Unpacking android-libziparchive:amd64 (1:34.0.5-13) over (1:34.0.5-12+b1)...
Preparing to unpack .../3-adb_1%3a34.0.5-13_amd64.deb...
Unpacking adb (1:34.0.5-13) over (1:34.0.5-12+b1)...
Preparing to unpack .../4-android-libcutils_1%3a34.0.5-13_amd64.deb...
Unpacking android-libcutils:amd64 (1:34.0.5-13) over (1:34.0.5-12+b1)...
Preparing to unpack .../5-android-libbase_1%3a34.0.5-13_amd64.deb...
Unpacking android-libbase:amd64 (1:34.0.5-13) over (1:34.0.5-12+b1)...
Preparing to unpack .../6-android-liblog_1%3a34.0.5-13_amd64.deb...
Unpacking android-liblog:amd64 (1:34.0.5-13) over (1:34.0.5-12+b1)...
Setting up android-liblog:amd64 (1:34.0.5-13)...
Setting up android-libbase:amd64 (1:34.0.5-13)...
Setting up android-libziparchive:amd64 (1:34.0.5-13)...
Setting up android-libcutils:amd64 (1:34.0.5-13)...
Setting up adb (1:34.0.5-13)...
Setting up android-libbacktrace:amd64 (1:34.0.5-13)...
Setting up android-libutils:amd64 (1:34.0.5-13)...
Processing triggers for kali-menu (2025.2.7)...
Processing triggers for libc-bin (2.42-16)...
Processing triggers for man-db (2.13.1-1)...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.
```

Figure 2.3: Installation of InsecureBankv2 using ADB.

2.5 Granting Application Permissions

After installation, the application requested multiple Android runtime permissions including access to storage, SMS, contacts, phone, and location. These permissions were granted to allow the application to function during testing.

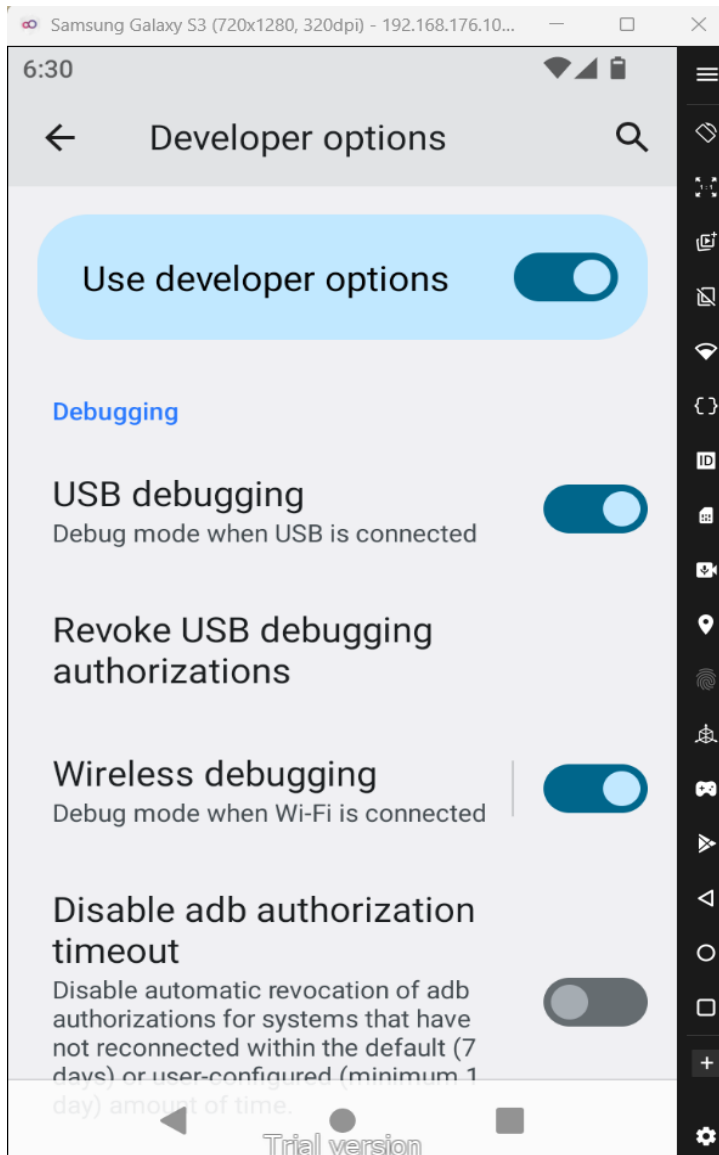


Figure 2.4: Runtime permissions granted to the application.

2.6 Launching the Application

The InsecureBankv2 application was successfully launched within the emulator. The login interface provided fields for username and password along with an Autofill Credentials option.

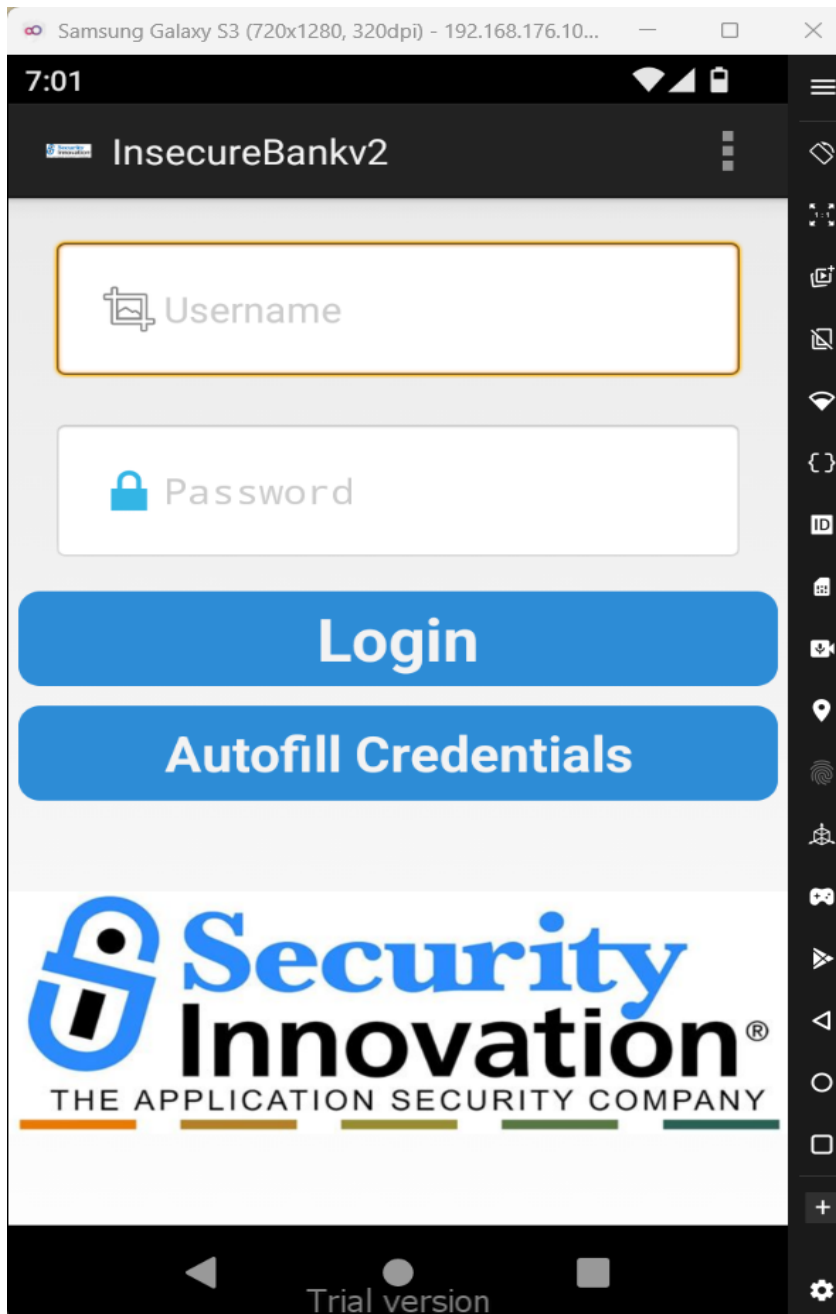


Figure 2.5: InsecureBankv2 login screen.

The Flask backend server was started from the **AndroLabServer** directory using the following command:

python app.py

The server successfully started on port **8888** and listened for incoming requests from the Android application.

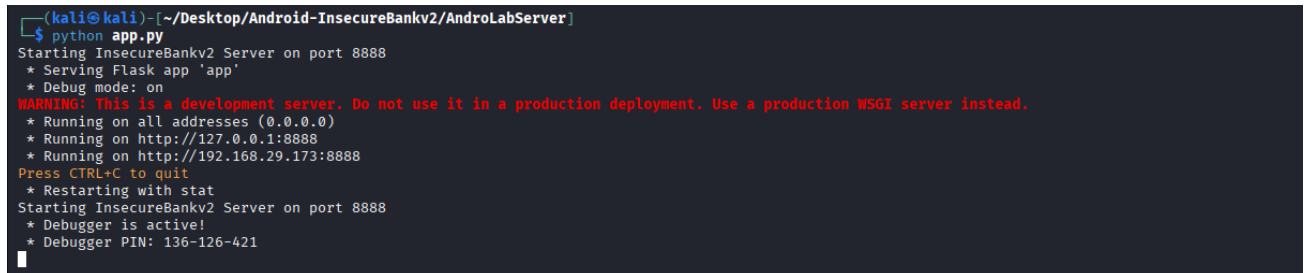


Figure 2.6: Flask backend server running successfully.

2.8 Burp Suite Configuration

Burp Suite Community Edition was configured as the interception proxy. A proxy listener was created on port **8080** to capture HTTP traffic generated by the Android application.

Intercept mode was enabled before launching the application.

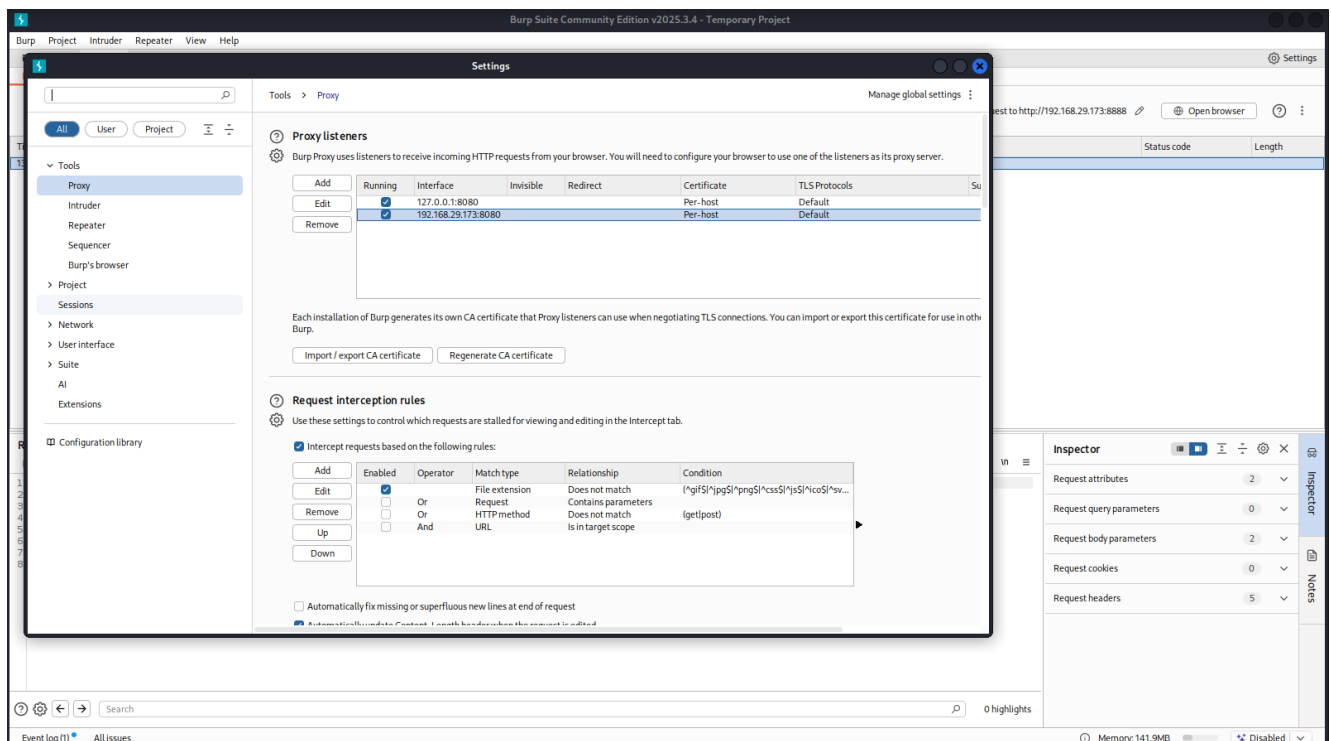


Figure 2.7: Burp Suite proxy listener configuration.

3. Vulnerability Testing

Vulnerability 1 – Insecure HTTP Communication

Definition

Applications transmitting sensitive information over HTTP instead of HTTPS expose user credentials to interception by attackers positioned on the same network.

Example

The InsecureBankv2 application communicated with the backend server using HTTP requests over port 8888.

Steps Followed

1. Started the Flask backend server.
2. Configured Burp Suite proxy listener.
3. Launched the Android application.
4. Attempted to log in.
5. Captured the login request in Burp Suite.

Screenshot

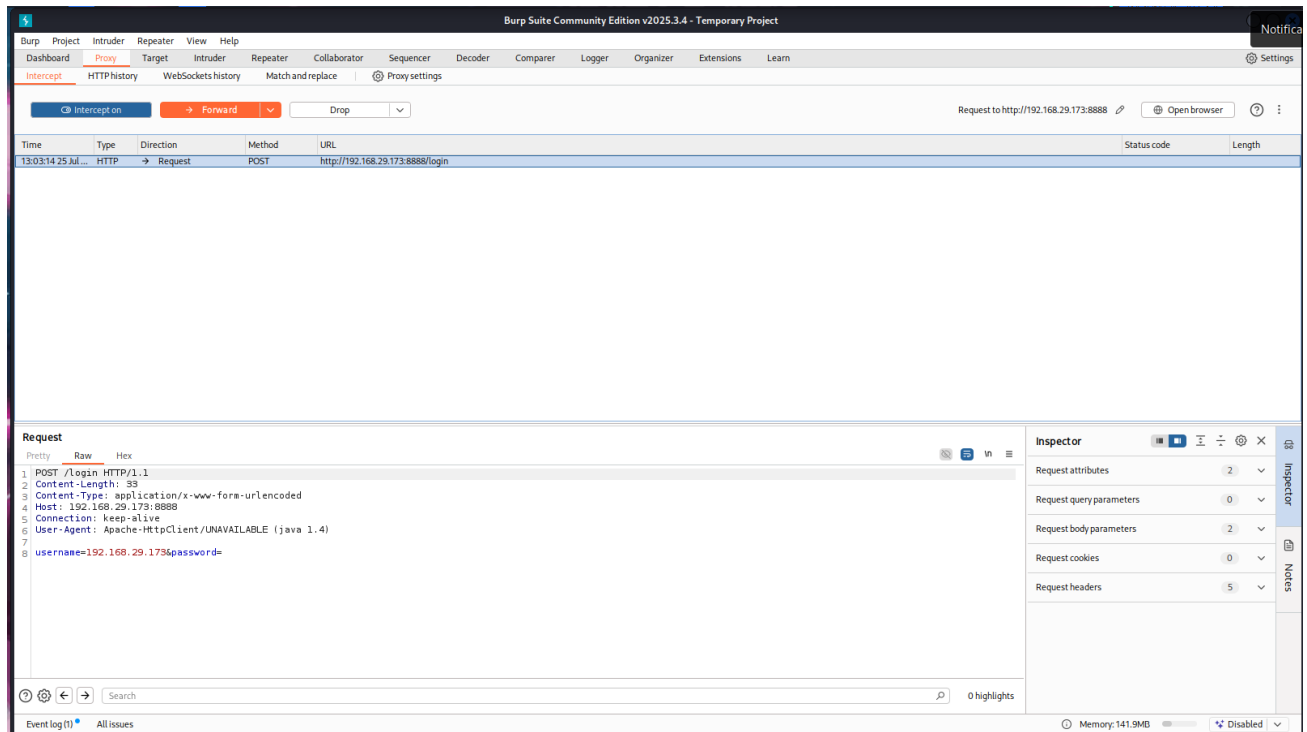


Figure 3.1: Login request intercepted using Burp Suite.

Observations

The intercepted request showed that the login credentials were transmitted over an unencrypted HTTP connection.

The request contained:

POST /login

username=192.168.29.173

password=

Since the communication used HTTP, it could be intercepted by anyone with access to the same network.

Tools Used

- Burp Suite Community Edition
- Genymotion
- Kali Linux

- Flask Server
-

Proof of Concept

The intercepted HTTP request successfully demonstrated that application traffic could be analyzed and modified before reaching the backend server.

Remediation

- Implement HTTPS using TLS.
 - Configure valid SSL certificates.
 - Enforce certificate validation.
 - Disable plaintext communication.
-

Vulnerability 2 – Information Disclosure

Definition

Applications should avoid revealing unnecessary authentication details through server responses because they may help attackers enumerate valid usernames or understand backend behavior.

Example

The backend server returned the following response:

```
{  
  "message": "User Does Not Exist",  
  "user": "192.168.29.173"  
}
```

Steps Followed

1. Submitted invalid login credentials.
2. Captured the server response.
3. Reviewed the backend server logs.

Screenshot

The screenshot displays the Burp Suite Community Edition v2025.3.4 interface. The top menu bar includes options like Burp, Project, Intruder, Repeater, View, and Help. Below the menu, the 'Proxy' tab is active, showing a list of intercepted HTTP history. The table below lists several intercepted requests, all of which are POST requests to the /login endpoint on the host 192.168.29.173. The status code for all requests is 200, and the length is 233 bytes. The MIME type is JSON, and the extension is .json. The title is /login. The notes column is empty. The TLS column is empty. The IP column is 192.168.29.173. The cookies column is empty. The time column shows the request time, and the listener port is 8080. The start response is 235.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response
1	http://192.168.29.173:8080	POST	/login		✓	200	233	JSON		/login			192.168.29.173		13:03:14.25 J...	8080	235
2	http://192.168.29.173:8080	POST	/login		✓	200	226	JSON		/login			192.168.29.173		13:07:01.25 J...	8080	8
3	http://192.168.29.173:8080	POST	/login		✓	200	226	JSON		/login			192.168.29.173		13:07:01.25 J...	8080	7
4	http://192.168.29.173:8080	POST	/login		✓	200	226	JSON		/login			192.168.29.173		13:07:01.25 J...	8080	11
5	http://192.168.29.173:8080	POST	/login		✓	200	226	JSON		/login			192.168.29.173		13:07:01.25 J...	8080	4
6	http://192.168.29.173:8080	POST	/login		✓	200	226	JSON		/login			192.168.29.173		13:09:44.25 J...	8080	7
7	http://192.168.29.173:8080	POST	/login		✓	200	226	JSON		/login			192.168.29.173		13:09:45.25 J...	8080	2
8	http://192.168.29.173:8080	POST	/login		✓	200	226	JSON		/login			192.168.29.173		13:09:45.25 J...	8080	3
9	http://192.168.29.173:8080	POST	/login		✓	200	226	JSON		/login			192.168.29.173		13:09:45.25 J...	8080	7
10	http://192.168.29.173:8080	POST	/login		✓	200	226	JSON		/login			192.168.29.173		13:09:45.25 J...	8080	4
11	http://192.168.29.173:8080	POST	/login		✓	200	226	JSON		/login			192.168.29.173		13:09:45.25 J...	8080	5
12	http://192.168.29.178:8080	POST	/login		✓					/login			192.168.29.178		13:10:42.25 J...	8080	
13	http://192.168.29.178:8080	POST	/login		✓					/login			192.168.29.178		13:10:45.25 J...	8080	

The main window shows the 'Request' and 'Response' tabs. The 'Request' tab is active, showing a POST request to /login HTTP/1.1. The request body is a JSON object: {"username": "192.168.29.173", "password": "192.168.29.173"}. The 'Response' tab is also active, showing a 200 OK response from the server. The response body is a JSON object: {"message": "User Does not Exist", "user": "192.168.29.173"}. The 'Inspector' panel on the right shows the request attributes, request body parameters, request headers, and response headers. The status bar at the bottom indicates that the memory usage is 133.9MB and the proxy is disabled.

Figure 3.2: Captured HTTP response.

```
(kali@kali)-[~/Desktop/Android-InsecureBankv2/AndroLabServer]
$ python app.py
Starting InsecureBankv2 Server on port 8888
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8888
* Running on http://192.168.29.173:8888
Press CTRL+C to quit
* Restarting with stat
Starting InsecureBankv2 Server on port 8888
* Debugger is active!
* Debugger PIN: 136-126-421
u = None
{"message": "User Does not Exist", "user": "192.168.29.173"}
192.168.29.173 - - [25/Jul/2026 13:07:00] "POST /login HTTP/1.1" 200 -
u = None
{"message": "User Does not Exist", "user": "android"}
192.168.29.173 - - [25/Jul/2026 13:07:01] "POST /login HTTP/1.1" 200 -
u = None
{"message": "User Does not Exist", "user": "android"}
192.168.29.173 - - [25/Jul/2026 13:07:01] "POST /login HTTP/1.1" 200 -
u = None
{"message": "User Does not Exist", "user": "android"}
192.168.29.173 - - [25/Jul/2026 13:07:01] "POST /login HTTP/1.1" 200 -
u = None
{"message": "User Does not Exist", "user": "android"}
192.168.29.173 - - [25/Jul/2026 13:09:44] "POST /login HTTP/1.1" 200 -
u = None
{"message": "User Does not Exist", "user": "android"}
192.168.29.173 - - [25/Jul/2026 13:09:44] "POST /login HTTP/1.1" 200 -
u = None
{"message": "User Does not Exist", "user": "android"}
192.168.29.173 - - [25/Jul/2026 13:09:45] "POST /login HTTP/1.1" 200 -
u = None
{"message": "User Does not Exist", "user": "android"}
192.168.29.173 - - [25/Jul/2026 13:09:45] "POST /login HTTP/1.1" 200 -
u = None
{"message": "User Does not Exist", "user": "android"}
192.168.29.173 - - [25/Jul/2026 13:09:45] "POST /login HTTP/1.1" 200 -
u = None
{"message": "User Does not Exist", "user": "android"}
192.168.29.173 - - [25/Jul/2026 13:09:45] "POST /login HTTP/1.1" 200 -
u = None
{"message": "User Does not Exist", "user": "android"}
192.168.29.173 - - [25/Jul/2026 13:09:45] "POST /login HTTP/1.1" 200 -
```

Figure 3.3: Backend server logs showing received requests.

Observations

The backend explicitly disclosed that the supplied username did not exist. Such detailed authentication responses can assist attackers in identifying valid user accounts.

Tools Used

- Burp Suite
 - Flask Server
-

Proof of Concept

The application returned detailed authentication responses rather than a generic login failure message.

Remediation

- Return generic authentication messages such as "Invalid username or password."
- Avoid exposing backend logic.
- Implement account lockout policies.
- Introduce rate limiting to reduce brute-force attempts.

4. Summary Table

Vulnerability	Tool Used	Evidence Screenshot	Status	Mitigation
Insecure HTTP Communication	Burp Suite	Burp Intercept Request	Vulnerable	Use HTTPS with TLS
Information Disclosure	Burp Suite, Flask	Burp Response & Flask Logs	Vulnerable	Return generic authentication errors

5. Conclusion

Summary of Learning

This project provided practical experience in configuring an Android penetration testing environment using Genymotion, Burp Suite Community Edition, Python Flask, and the vulnerable InsecureBankv2 application. The testing process demonstrated how mobile application traffic can be intercepted, analyzed, and documented.

Two security issues were observed during testing: insecure HTTP communication and information disclosure through detailed authentication responses. These findings emphasize the importance of implementing secure communication channels and minimizing information exposed by backend services.

Limitations

- Testing was limited to the features provided by the InsecureBankv2 application.
- Static source code analysis was not performed.
- Reverse engineering of the APK was outside the scope of this project.
- Advanced mobile testing techniques such as SSL pinning bypass, runtime instrumentation, and root detection bypass were not included.

Future Improvements

Future work may include:

- Performing static analysis using MobSF.
- Testing additional OWASP Mobile vulnerabilities.
- Evaluating SSL/TLS implementation and certificate pinning.
- Using Frida or Objection for dynamic runtime analysis.
- Automating vulnerability scanning for Android applications.

6. References

1. OWASP Mobile Security Testing Guide (MSTG) – <https://owasp.org/www-project-mobile-security-testing-guide/>
2. OWASP Mobile Top 10 – <https://owasp.org/www-project-mobile-top-10/>
3. Burp Suite Documentation – <https://portswigger.net/burp/documentation>
4. Android Developers Documentation – <https://developer.android.com/docs>
5. Genymotion Documentation – <https://docs.genymotion.com/>
6. Flask Documentation – <https://flask.palletsprojects.com/>
7. Android Debug Bridge (ADB) Documentation – <https://developer.android.com/tools/adb>
8. InsecureBankv2 GitHub Repository – <https://github.com/dineshshetty/Android-InsecureBankv2>